PROJECT SOFTWARE DESIGN SPECIFICATION

Project Title: Engineering Resource Management System (ERMS)

Document Version: 1.0

Date: 2026-04-08

Prepared By: ERMS Core Platform Team

Organization/Department: Mahindra University, Software Engineering Course

Reviewed By: Prof. Kg Krishna / Avinash Arun Chauhan

Approved By: Prof. Kg Krishna / Avinash Arun Chauhan

Document Control

Revision History

Version	Date	Author	Description of Change
0.1	2026-04-08	ERMS Core Platform Team	Initial SDS prepared from submitted SRS v1.3
1.0	2026-04-08	ERMS Core Platform Team	Completed design baseline aligned to implemented modules

Distribution List

Name	Role	Department/Organization
ERMS Core Platform Team	Development Team	Software Engineering, Mahindra University
Prof. Kg Krishna / Avinash Arun Chauhan	Instructor/Reviewer	Software Engineering, Mahindra University
Sravanthi Acchugatla	Teaching Assistant	Software Engineering, Mahindra University

Authors

Name	Student ID
Abhishek Gupta	SE23UCSE011
Mohammed Sadath	SE23UCSE230
Faizan Ahmed	SE23UCSE003
Jai Sai Karthik Bonam	SE23UCSE040
Sharon Benhur	SE23UCSE036
Gayatri Ravinder	SE23UARI039

Contents

Chapter 1

Introduction

1.1 Purpose

This Software Design Specification (SDS) defines the software design of the Engineering Resource Management System (ERMS), a web-based lab inventory and equipment management platform. It translates the approved SRS into a design-level blueprint for implementation, integration, validation, and maintenance.

1.2 Scope

ERMS provides centralized workflows for inventory lifecycle management, item requests, borrowed equipment tracking, supplier management, alerts, and analytics. This SDS covers architecture, component responsibilities, interfaces, data contracts, non-functional design, and validation strategy for the current implementation baseline (frontend complete; backend core modules in progress).

1.3 Definitions, Acronyms, and Abbreviations

- **SDS**: Software Design Specification
- **SRS**: Software Requirements Specification
- **ERMS**: Engineering Resource Management System
- **API**: Application Programming Interface
- **RBAC**: Role-Based Access Control
- **SPA**: Single-Page Application

- **DRF**: Django REST Framework

1.4 References

- SRS v1.3 for ERMS (dated 2026-04-06)
- IEEE SRS Guidelines
- UML Modeling Standards
- COMET Software Design Method

1.5 Document Overview

The document starts with system context and goals, then defines architecture and detailed component design. It then specifies data and interfaces, captures non-functional design decisions, and concludes with testing approach and appendices.

Chapter 2

System Overview

2.1 System Context

ERMS is deployed as a browser-accessible SPA for students, faculty, and administrators. The frontend communicates with a Django REST API. The backend persists data in PostgreSQL, uses Redis for caching, and integrates OpenAI-backed services for AI insight generation. Docker Compose orchestrates local and production-like environments.

2.2 Design Goals and Constraints

- Maintain a modular monorepo architecture with clear separation between frontend and API services.
- Meet responsiveness goals: CRUD endpoints within 2 seconds; dashboard/analytics views within 3 seconds under normal load.
- Support at least 200 concurrent users for common web interactions.
- Enforce production authentication and RBAC (admin vs user), replacing current local frontend session guards.
- Conform to defined stack: React + Vite + TypeScript, Tailwind CSS, shadcn-ui, React Query, Django DRF, PostgreSQL, Redis, Docker.

Chapter 3

Architectural Design

3.1 Architecture Summary

ERMS adopts a layered web architecture:

- Presentation layer: React SPA with route-based modules (Dashboard, Inventory, Requests, Borrowed, Suppliers, Reports, Activity, AI Insights, Analytics, Alerts).
- Application/API layer: DRF modules exposing REST endpoints per domain.
- Data layer: PostgreSQL for durable transactional records and Redis for caching/faster repeated reads.
- AI/RAG layer: OpenAI-backed query and insight generation, persistent knowledge chunks, and hybrid retrieval over operational data.
- Integration/deployment layer: Dockerized services with API, cache, database, and frontend containers.

This architecture supports independent domain evolution, easier testing boundaries, and clear ownership across team roles.

3.2 Subsystem Decomposition

Subsystem	**Responsibility**
Frontend SPA	UI rendering, route guards, forms, filtering, module navigation, API consumption via React Query

Inventory Service	CRUD for inventory items, category/location/supplier-linked records, stock threshold support
Requests Service	Item request creation and lifecycle transitions: Pending, Approved/Rejected, Issued
Borrowed Service	Borrow/return lifecycle management, overdue detection, borrower tracking
Suppliers Service	Supplier profile CRUD and relationship to inventory items
Alerts and Activity Services	Event feed and alert lifecycle operations: active, acknowledged, resolved, dismissed
AI and Analytics Services	Insight generation and KPI/stock-health endpoint aggregation
Data Layer	PostgreSQL persistence, Redis caching, data integrity and auditability support

3.3 Deployment Architecture

Deployment uses containerized services orchestrated with Docker Compose:

- Frontend container serving bundled SPA assets.
- API container running Django/DRF business services.
- PostgreSQL container for relational persistence.
- Redis container for cache and fast lookups.
- Reverse proxy and production compose manifests for deployment profiles.

Chapter 4

Detailed Design

4.1 Component Design

4.1.1 Component: Inventory Module

- Responsibility: Maintain inventory item master records and stock state.
- Inputs: Item metadata (name, category, quantity, location, supplier, notes, valuation fields).
- Outputs: Updated item records, low-stock indicators, item-level updates visible across dashboard/analytics.
- Dependencies: Suppliers module, PostgreSQL, activity logging.

4.1.2 Component: Request Workflow Module

- Responsibility: Create and process item requests through controlled status transitions.
- Inputs: Request form data (item, quantity, reason, requester).
- Outputs: Request records with lifecycle states and issuance events.
- Dependencies: Inventory availability checks, role permissions, activity/alerts.

4.1.3 Component: Borrowed Equipment Module

- Responsibility: Track physical borrow operations and returns.
- Inputs: Borrower details, borrow date, expected return date, item reference.

- Outputs: Borrow records, overdue flags, return status updates.
- Dependencies: Inventory module, periodic overdue checks, alerts center.

4.1.4 Component: Supplier Module

- Responsibility: Maintain supplier profiles and support procurement context in inventory.
- Inputs: Supplier identity/contact/address details and history attributes.
- Outputs: Supplier records consumable by inventory and analytics.
- Dependencies: PostgreSQL, inventory relationships.

4.1.5 Component: Dashboard, Reports, Activity, Alerts, AI/Analytics

- Responsibility: Provide operational visibility, event timeline, risk signaling, and decision support.
- Inputs: Aggregated domain data and AI service outputs.
- Outputs: KPI tiles/charts, reports, activity entries, actionable alerts, AI suggestions.
- Dependencies: All core backend services, cache, OpenAI integration.

4.1.6 Component: AI Intelligence and RAG Module

- Responsibility: Provide natural-language Q&A, automated insight generation, reorder simulation, and human-reviewed AI suggestions.
- Inputs: Live operational data from inventory, requests, borrowed records, suppliers, alerts, and activity.
- Outputs: AI answers with confidence/citations, insight records, suggestion workflows, and AI usage telemetry.
- Dependencies: OpenAI API client, persistent KnowledgeChunk index, prompt templates, API throttling, and audit models.

4.2 Data Design

4.2.1 Data Model

Primary entities are InventoryItem, ItemRequest, BorrowedItem, Supplier, ActivityEntry, AlertRecord, AIInteraction, InsightRecord, AISuggestion, and KnowledgeChunk. Core relationships:

- Supplier (1) to InventoryItem (N)
- InventoryItem (1) to ItemRequest (N)
- InventoryItem (1) to BorrowedItem (N)
- Domain actions (request, borrow, issue, return, stock change) to ActivityEntry (N)
- Inventory/Supplier/risk events to AlertRecord (N)
- AIInteraction (1) to InsightRecord (N) and AISuggestion (N) for auditability of generated outputs
- KnowledgeChunk stores RAG corpus entries keyed by operational source with embedding vectors and metadata

UUID identifiers are used across API contracts for interoperability and traceability.

4.2.2 Database Schema

4.3 Interface Design

4.3.1 User Interface

The frontend is a SPA with persistent sidebar navigation and protected routes. Main screens:

- Dashboard with key metrics and overdue/pending indicators.
- Inventory, Requests, Borrowed Equipment, Suppliers CRUD workflows.
- Reports and Analytics views for KPI and stock health.

Field	Type	Description
inventory_item.id	UUID	Inventory item identifier
item_request.status	ENUM	Pending, Approved, Rejected, Issued
borrowed_item.status	ENUM	Active, Returned, Overdue
supplier.id	UUID	Supplier identifier
alert_record.status	ENUM	active, acknowledged, resolved, dismissed
activity_entry.timestamp	DATETIME	Domain event timestamp
ai_interaction.total_tokens	INTEGER	Token accounting for each AI call
insight_record.severity	ENUM	info, warning, critical
ai_suggestion. approval_status	ENUM	pending, approved, rejected, executed
knowledge_chunk.embedding	JSON	Vector embedding used for semantic retrieval

- Activity timeline for event visibility.
- Alerts Center with filtering and acknowledge/resolve/dismiss actions.
- AI Insights panel for natural-language queries and suggestions.

4.3.2 External Interfaces

- REST endpoints exposed by DRF modules for inventory, requests, borrowed, suppliers, alerts, activity, analytics, and AI.
- AI endpoints: `/api/ai/query/`, `/api/ai/insights/`, `/api/ai/simulate-reorder/`, `/api/ai/suggestions/`, `/api/ai/usage/`.
- Analytics endpoints: `/api/analytics/kpi/`, history, movements, demand signals, supplier performance, and stock-health.
- Alerts endpoints: `/api/alerts/`, `/api/alerts/risks/`, and `/api/alerts/compute/`.
- PostgreSQL interface via ORM/data access layer.
- Redis interface for cache-backed query acceleration.
- OpenAI API integration for insight generation.
- Browser interface for user access and interaction.

4.4 Algorithm Design

Key processing logic includes:

- Request lifecycle state transition rules and issuance-triggered quantity decrement.
- Borrow lifecycle with overdue determination based on expected return date and unresolved return status.
- Alert engine computes stockout, overstock, and supplier risk scores and maps score bands to info/warning/critical alerts.
- KPI/analytics aggregation computes live health score, fill rate, inventory turnover, stock-health buckets, and supplier metrics.
- RAG indexing pipeline builds KnowledgeChunk entries from live operational tables, persists embeddings, and deactivates stale chunks.
- Hybrid retrieval combines semantic similarity (embeddings) and lexical ranking to ground AI responses with citations.
- AI request controls apply throttles (`ai_insights_read`, `ai_write`) and capture token/cost/latency audit data.

Chapter 5

Non-Functional Design

5.1 Performance

The design targets CRUD API responses within 2 seconds under normal load. Dashboard, analytics, and alerts views should render within 2 seconds for typical datasets. System capacity target is at least 200 concurrent web users. OpenAI-backed AI endpoints are rate-limited (`ai_write`, `ai_insights_read`) to control latency and cost under load.

5.2 Security

Production deployment must enforce backend authentication and protected route access with RBAC. Administrative actions (inventory, supplier records, request approval, alert actions) are restricted by role. Secrets (database credentials, OpenAI and third-party API keys) are environment-driven and never committed to source control. Data protection in transit and at rest is required by deployment policy.

5.3 Reliability and Availability

Service design uses modular APIs, durable relational storage, and containerized restarts for operational resilience. Activity and lifecycle logging support operational auditability. Backup and recovery policy is aligned with PostgreSQL container/storage configuration in deployment.

5.4 Maintainability

Monorepo organization and domain-separated backend modules simplify onboarding and refactoring. Typed frontend modules (TypeScript), DRF structure, DRY discipline across shared clients and serializers, and explicit API contracts improve maintainability and testability. Planned monitoring and expanded automated tests support long-term evolution.

Chapter 6

Testing and Validation

6.1 Verification Strategy

Validation follows layered testing:

- Unit tests for model logic, state transitions, and utility functions.
- Integration tests for API endpoints and database interactions.
- UI flow tests for request, borrow, inventory, and alert workflows.
- Acceptance checks mapped to key SRS functional requirements (F1–F8).
- AI and analytics validation: OpenAI fallback behavior, RAG retrieval grounding, insight and suggestion API contracts, and throttle handling (HTTP 429) in the frontend.

6.2 Test Environment

Testing is executed in the project containerized environment using the same service layout as development: frontend, API, PostgreSQL, and Redis. Representative seed data is used for inventory categories, supplier records, request states, borrow records, and alert conditions.

6.3 Traceability Matrix

Requirement ID	**Design Element**	**Test Case ID**
F1 Inventory Management	Inventory Module Design	TC-INV-001
F2 Request Workflow	Request Workflow Module	TC-REQ-001
F3 Borrow Tracking	Borrowed Equipment Module	TC-BRW-001
F4 Supplier Management	Supplier Module	TC-SUP-001
F5 Dashboard and Reports	Dashboard/Reports Components	TC-RPT-001
F6 Auth and Access Control	Security Design and Route Guards	TC-AUTH-001
F7 Alerts Management	Alerts Service Design	TC-ALT-001
F8 AI Insights and Analytics	AI and Analytics Services	TC-AI-001

Chapter 7

Appendices

7.1 Appendix A: Diagrams

Final UML artifacts:

- High-level architecture diagram
- Request/borrow sequence diagrams
- Class/domain model for core entities

7.2 Appendix B: Open Issues

Open design actions aligned to the SRS next steps:

- Complete backend auth and RBAC enforcement.
- Expand automated test coverage for critical workflows.
- Harden API validation and production error handling.
- Finalize observability and monitoring baseline.

7.3 Appendix C: Glossary

ERMS domain terms:

- **Issued**: approved request with stock allocated and recorded.
- **Overdue**: borrowed item not returned by expected return date.

- **Stock Health**: aggregate indicator of quantity and threshold risk.
- **Alert Lifecycle**: active, acknowledged, resolved, dismissed.